

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS2DA: Data Structures & Algorithms

Master Theorem and Trees

Recap

- Big-O notation — focus on fastest growing term
 - Gives summary of time/space usage independent of algorithm implementation details.
 - Focuses on whether algorithm scales well for large inputs.
 - Ignores "constant factors" — even though they are important in practice.
 - Is an upper bound — an $\mathcal{O}(n)$ algorithm is also $\mathcal{O}(n^2)$.
 - Can be determined by solving recurrence equations.
- Divide & Conquer algorithms
 - Mergesort — sorting in guaranteed $\mathcal{O}(n \log n)$ time, but $\mathcal{O}(n)$ space
 - Quickselect — find order statistics in average $\mathcal{O}(\log n)$ time
 - Mergehull — convex hull in $\mathcal{O}(n \log n)$ time including preprocessing sort

Today

- Divide & Conquer — integer multiplication
- Master Theorem — solving common recurrences
- Trees
 - Tree Traversals
 - Binary Search Trees
 - AVL Trees

"Faster" Integer Multiplication

Integer Multiplication: Karatsuba Algorithm

Suppose we want to multiply binary integers x and y .

We can write them as:

- $x = (x_1 * 2^n) + x_0$
- $y = (y_1 * 2^n) + y_0$

where x_0, y_0 are low n bits and x_1, y_1 are high bits. Then:

$$\begin{aligned}xy &= ((x_1 * 2^n) + x_0)((y_1 * 2^n) + y_0) \\&= (x_1 * y_1 * 2^{2n}) + (((x_1 * y_0) + (x_0 * y_1)) * 2^n) + (x_0 * y_0) \\&= (z_2 * 2^{2n}) + (z_1 * 2^n) + z_0\end{aligned}$$

where:

- $z_2 = x_1 * y_1$
- $z_1 = (x_1 * y_0) + (x_0 * y_1)$
- $z_0 = x_0 * y_0$

Integer Multiplication: Karatsuba Algorithm

We have:

- $z_2 = x_1 * y_1$
- $z_1 = (x_1 * y_0) + (x_0 * y_1)$
- $z_0 = x_0 * y_0$

Rewrite z_1 :

$$\begin{aligned} z_1 &= (x_1 * y_0) + (x_0 * y_1) \\ &= (x_1 + x_0)(y_0 + y_1) - (x_1 * y_1) - (x_0 * y_0) \\ &= z_3 - z_2 - z_0 \end{aligned}$$

where:

- $z_3 = (x_1 + x_0) * (y_0 + y_1)$

We can calculate z_1 with only one multiplication.

Integer Multiplication: Karatsuba Algorithm

Putting it all together, using 3 smaller multiplications for one big one:

Input: Integers x and y , length in bits b

Output: Product of x and y

```
if  $b < 32$   -- assuming 32 is machine word size
    return  $x*y$ 
```

```
split  $x$  into  $x_1, x_0$  with bitshifts
split  $y$  into  $y_1, y_0$  with bitshifts
```

```
 $z_3 = \text{multiply}(x_1 + x_0, y_0 + y_1, b/2)$   -- Addition of  $b$ -bit numbers is  $O(b)$ 
 $z_2 = \text{multiply}(x_1, y_1, b/2)$ 
 $z_0 = \text{multiply}(x_0, y_0, b/2)$ 
 $z_1 = z_3 - z_2 - z_0$ 
```

```
return  $(z_2 \ll 2*(b/2)) + (z_1 \ll b/2) + z_0$ 
```

Master Theorem

We can solve many common recurrence equations using the **Master Theorem**.

Suppose: $T(n) = aT(\frac{n}{b}) + \mathcal{O}(n^r)$

Let: $c = \log_b a$ — the **critical exponent**

If	Then	Because
$r < c$	$T(n) = \mathcal{O}(n^c)$	cost of branching dominates
$r = c$	$T(n) = \mathcal{O}(n^c \log n)$	costs are balanced
$r > c$	$T(n) = \mathcal{O}(n^r)$	cost of combining dominates

(The $r = c$ case can be refined further.)

Analysis: Karatsuba Algorithm

We used 3 multiplications, plus $\mathcal{O}(n)$ work to do additions, so:

$$T(n) = 3T\left(\frac{n}{2}\right) + \mathcal{O}(n)$$

Thus:

- $a = 3$
- $b = 2$
- $r = 1$
- $c = \log_2 3 \approx 1.58$

As $r < c$, use 1st case of Master Theorem:

- $T(n) = \mathcal{O}(n^c) \approx \mathcal{O}(n^{1.58})$

Karatsuba's Algorithm is rarely used in practice.

Matrix Multiplication: Obvious Algorithm

To multiply square matrices:

Input: Square $n \times n$ matrices A and B

Output: Matrix $C = AB$

initialise C to all zeroes

for i from 1 to n // compute entries in each of n rows

 for j from 1 to n // compute n entries in row i

 for k from 1 to n

$C[i][j] = C[i][j] + A[i][k] * B[k][j]$

return C

Running time is $\mathcal{O}(n^3)$. Can we do better?

Matrix Multiplication: Strassen's Algorithm

Use a similar trick to Karatsuba's algorithm — but more complex.

See tutorial sheet for Week 3.

Binary Trees and Tree Traversals

Trees

Data often has a **tree** structure, where elements have **parents** and **children**:

- Arithmetic expressions
- Syntax of program source code
- Components in a GUI or webpage
- Settings stored in the Windows registry
- XML documents

Terminology:

- **Node**: data element in a tree
- **Root node**: unique node with no parent
- **Leaf node**: any node with no children
- **Branch node**: any node with children
- **Binary tree**: tree where nodes have ≤ 2 children — left child and right child

Expression Trees

Binary trees can encode arithmetic expressions.

Example: $(2 + 3) * (10 - 5)$

In expression trees:

- Branch nodes are operators
- Leaf nodes are operands
- No brackets needed — grouping is explicit in tree structure

Traversals: In-Order

There are standard ways of visiting the nodes in a tree, called **traversals**.

```
In-order traversal
```

```
Input: Root node of binary tree
```

```
if the node has a left child, traverse it
```

```
process node
```

```
if the node has a right child, traverse it
```

Output depends on exactly what the "process" is.

Printing Expression Trees

Use in-order traversal to print expression trees:

```
Print expression tree
Input: Expression tree
Output: Printed expression

if the node has a left child:
    print "("
    traverse left child
print node's data
if the node has a right child:
    traverse right child
    print ")"
```

(Avoiding redundant parentheses is harder.)

Try it.

Pre-Order Traversal

Pre-order traversal

Input: Root node of binary tree

process node

if the node has a left child, traverse it

if the node has a right child, traverse it

Post-Order Traversal

Post-order traversal

Input: Root node of binary tree

if the node has a left child, traverse it

if the node has a right child, traverse it

process node

Reverse Polish Notation

Print expression tree in Reverse Polish Notation

Input: Expression tree

Output: Printed expression in RPN

if the node has a left child, traverse it

if the node has a right child, traverse it

print the node's data

Try it.

Because operands can only occur at leaves and operators can only occur at branches, RPN uniquely determines grouping of expressions without using parentheses.

Stack Based Expression Calculator

Evaluate expression tree

Input: Expression tree

Output: Value of expression

create empty stack

traverse(root)

return top of stack

traverse(node):

 if the node has a left child, traverse it

 if the node has a right child, traverse it

 if the node's data is an operand (number):

 push the value onto a stack

 else node's data is an operator @:

 right = pop stack

 left = pop stack

 push left @ right

Try it.

Creating Expression Trees

Can we take an expression (perhaps without explicit parentheses) and generate an expression tree?

That is, given `(2 + 3) * (10 - 5)`, generate its expression tree?

Yes — this is called **parsing**.

In compilers, the most popular algorithm is **LALR(1)**, which runs in $\mathcal{O}(n)$ time.

For more complex **grammars**, such as fragments of English and other natural languages, **dynamic programming** algorithms such as **CYK** and **Earley's algorithm** run in $\mathcal{O}(n^3)$ time.

Binary Search Trees

Hash Tables Aren't Always Enough

Hash tables implement maps with $\mathcal{O}(1)$ insert, lookup and delete under **reasonable assumptions**.

Can we get fast maps without assumptions?

Yes — with **balanced** binary search trees — but we have to work for it.

We also get other operations, such as order statistics (minimum, maximum, median) or finding all keys/values in a range.

Binary Search Trees

Binary search trees use binary trees to store a set or map:

- Nodes store a **key** and its associated **value**.
- Keys must be **comparable**, so we can tell whether $a < b$, $a = b$ or $a > b$.
- For a branch node with key k :
 - All keys in left subtree are $< k$.
 - All keys in right subtree are $> k$.
- In-order tree traversal gives keys in ascending order.

Insert

Input: root node n, key k, value v

Output: root node, with tree now containing k

```
if n == null
    return new node(k, v)
if k < key(n)
    n.left = put(n.left, k, v)
if k > key(n)
    n.right = put(n.right, k, v)
if k == key(n)
    n.value = v
return n
```

Try it: 8 2 0 7 1 3 9 5 6 4

Lookup

Input: root node n, key k

Output: node containing k

```
if n == null
    return null
if k < key(n)
    return get(n.left, k)
if k > key(n)
    return get(n.right, k)
if k == key(n)
    return n
```

Pre-Order

Pre-order tree traversal has comparatively few uses.

However, using it to list the elements of a binary search tree gives the order needed to reconstruct the same tree using repeated insertions.

Delete

Deletion is easy if the node is a leaf or has only a single child; hard if two children.

Input: root node n, key k

Output: root node, with k deleted from tree

```
if n == null
    return null

if k < key(n)
    n.left = delete(n.left, k)
if k > key(n)
    n.right = delete(n.right, k)
if k == key(n)
    if n.right == null
        return n.left
    if n.left == null
        return n.right
    tmp = n;
    n = min(tmp.right)
    n.left = tmp.left
    n.right = delete_min(tmp.right)
return n

min(n):
    if n.left == null
        return n
    return min(n.left)

delete_min(n):
    if n.left == null
        return n.right
    n.left = delete_min(n.left)
    return n

-- Swap branch with right child's
-- left-most descendant.

-- Then delete that.
```

Balance and Complexity

Can we get fast maps without assumptions?

Yes — with **balanced** binary search trees — but we have to work for it.

Most binary search tree operations take time $O(h)$, where h is the height of the tree.

With keys inserted in random order, h is **expected** to be $O(\log n)$, but with keys inserted in sorted order **could be** $O(n)$.

In a balanced tree, all leaves are at **roughly** the same depth.

Maintaining this property takes extra work.

AVL (Adelson-Velsky and Landis) Trees

AVL trees are binary search trees that record **height** and maintain **balance**.

The height of a node's left and right subtrees may differ by at most one.

This ensures height is $\mathcal{O}(\log n)$. Hence all operations are $\mathcal{O}(\log n)$.

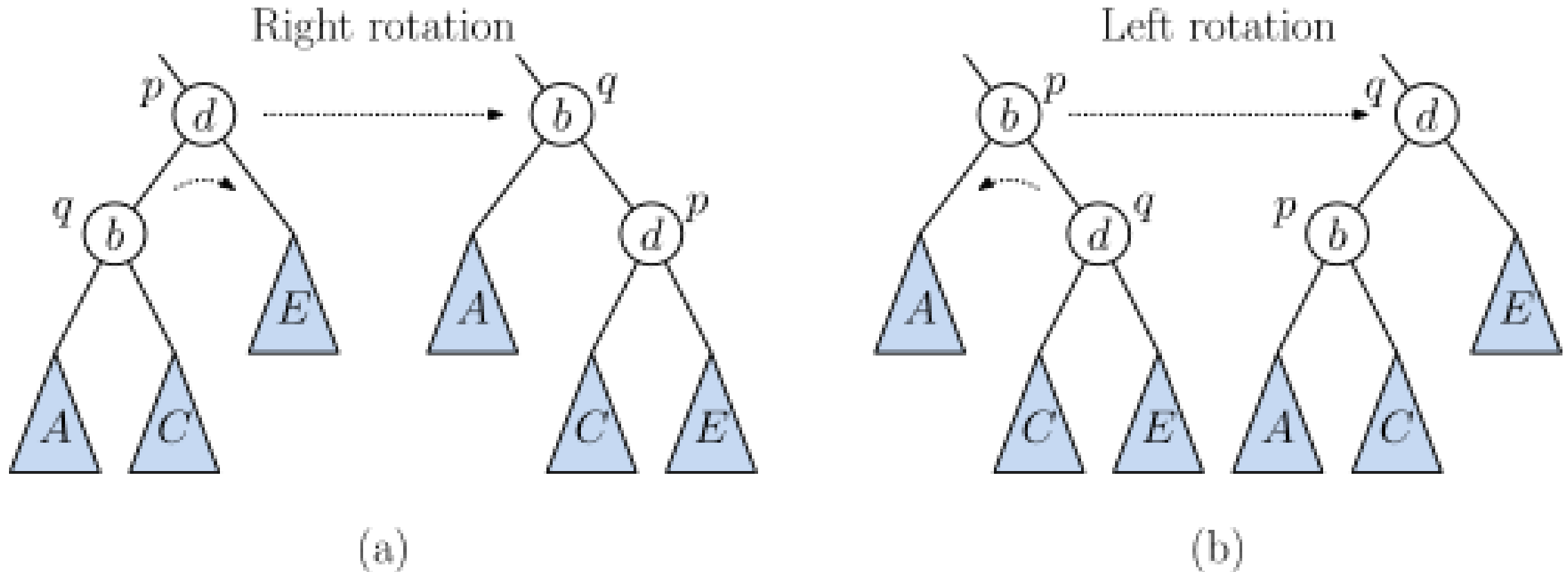
Operations on a tree are modified to recalculate subtree height and restore balance:

- **Lookup** — as for binary search tree
- **Insert** — **recalculate** height and **rebalance** with 1-2 **rotations**
- **Delete** — **recalculate** height and **rebalance** with $\mathcal{O}(\log n)$ **rotations**

Tree Rotations

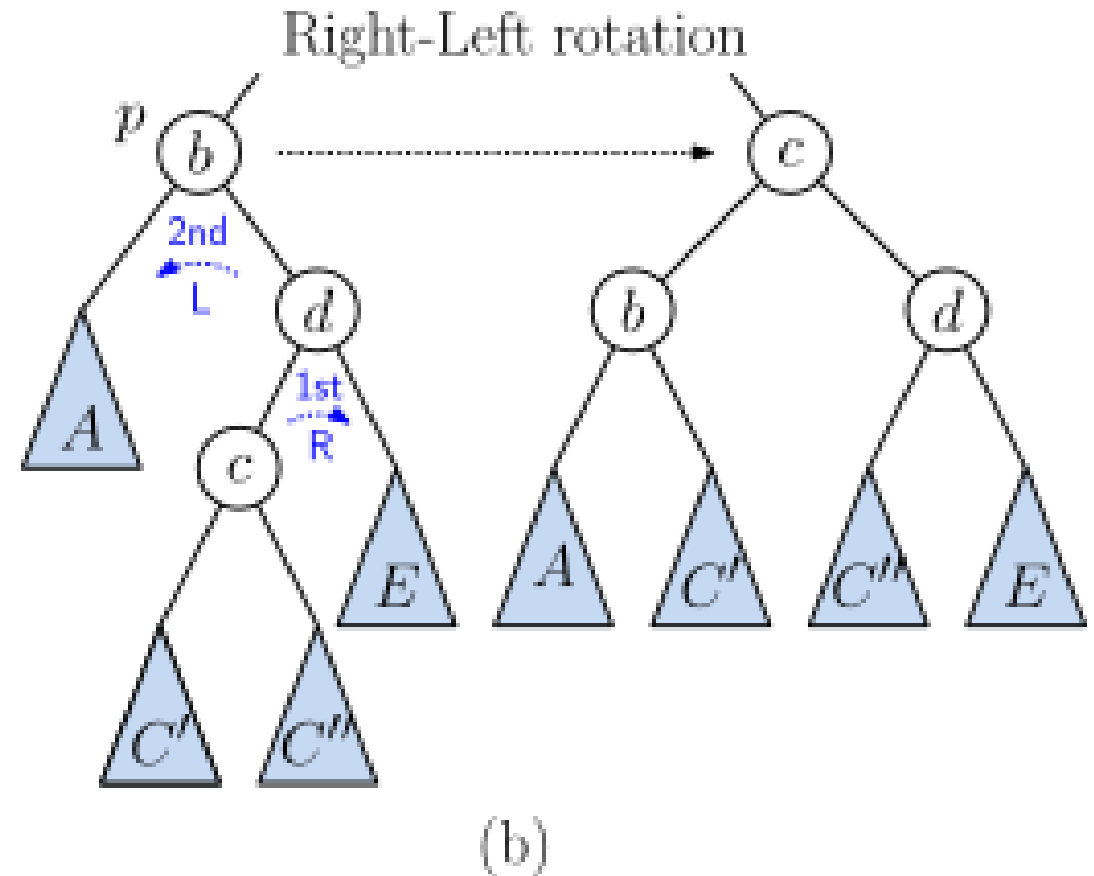
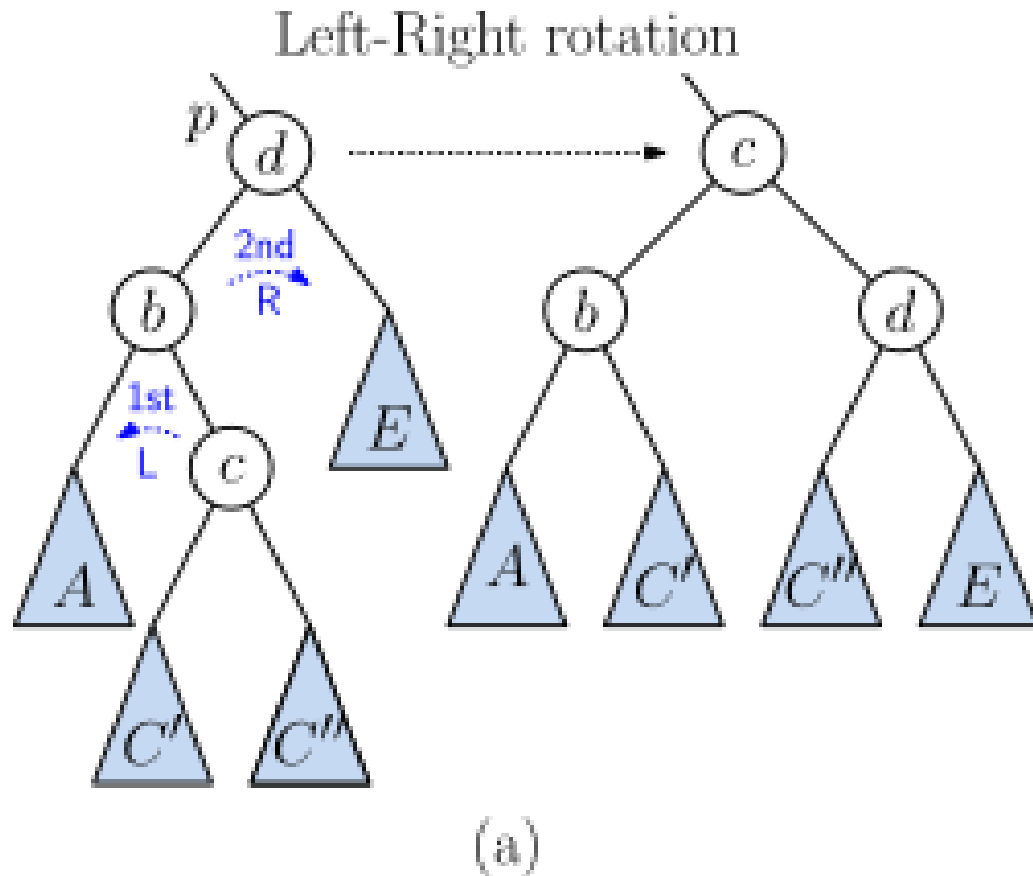
Tree rotations change the structure of a binary search tree, but maintain order of keys.

Left/Right Rotation



The height of the tree may change. With careful use, we can **rebalance** a tree.

Double Rotation



Source: Dave Mount, University of Maryland:

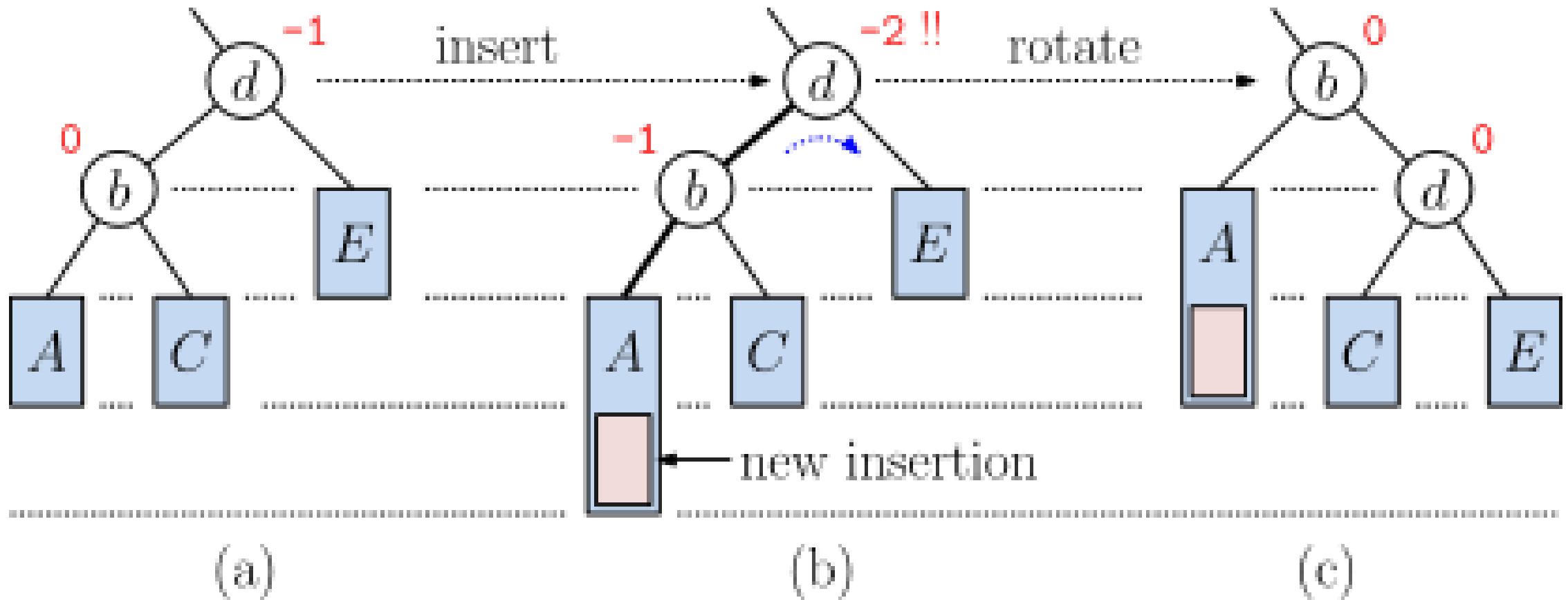
<https://www.cs.umd.edu/class/spring2022/cmsc420-0101/Lects/lect05-avl.pdf>

Rebalancing

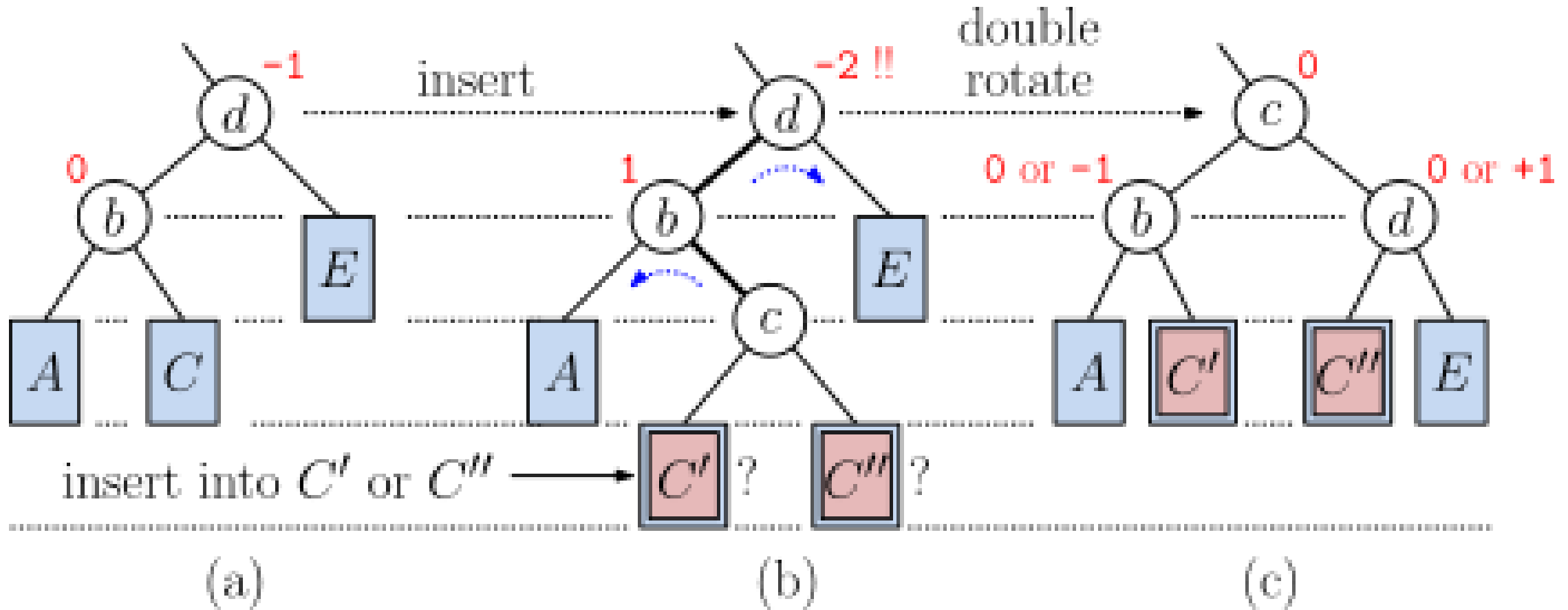
After insert/delete, travel back up the tree to root, recalculating node height and (if needed) rebalancing at every node:

```
private Node balance(Node x) {  
    if (balanceFactor(x) < -1) {  
        if (balanceFactor(x.right) > 0) {  
            x.right = rotateRight(x.right);  
        }  
        x = rotateLeft(x);  
    }  
    else if (balanceFactor(x) > 1) {  
        if (balanceFactor(x.left) < 0) {  
            x.left = rotateLeft(x.left);  
        }  
        x = rotateRight(x);  
    }  
    return x;  
}
```

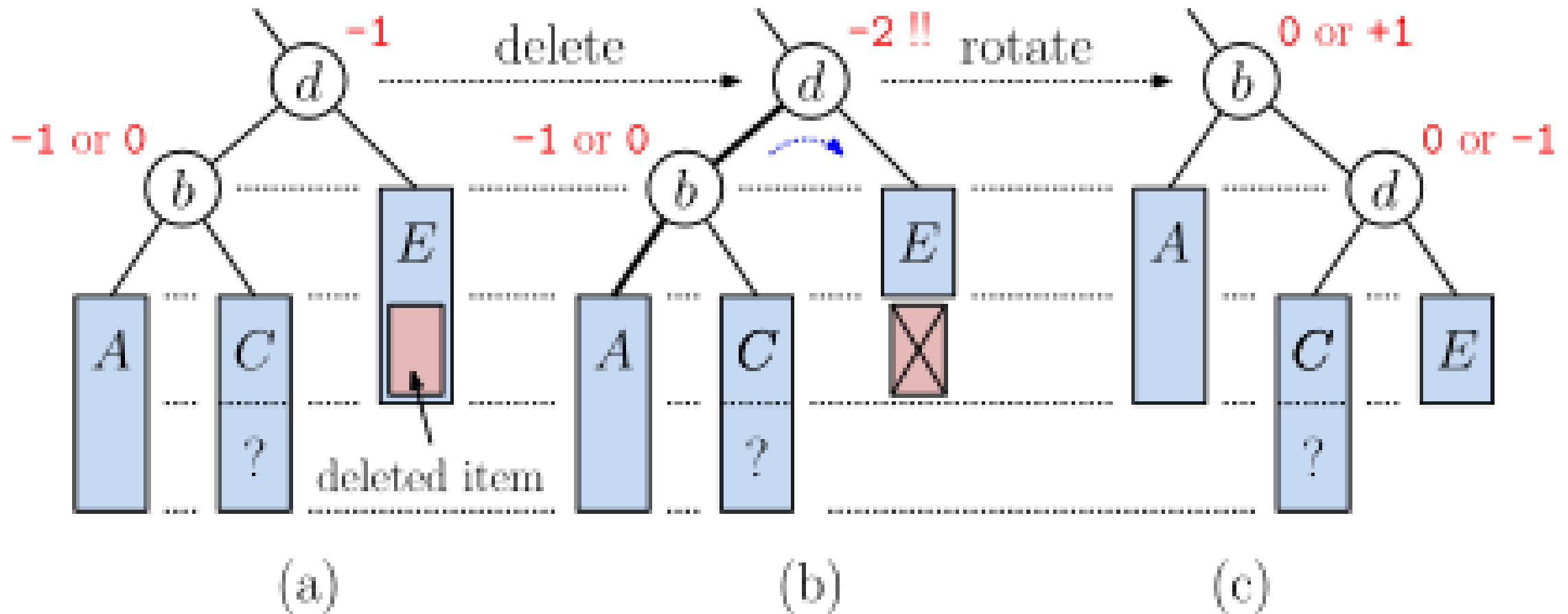
Rebalancing after Insert (Single Rotation)



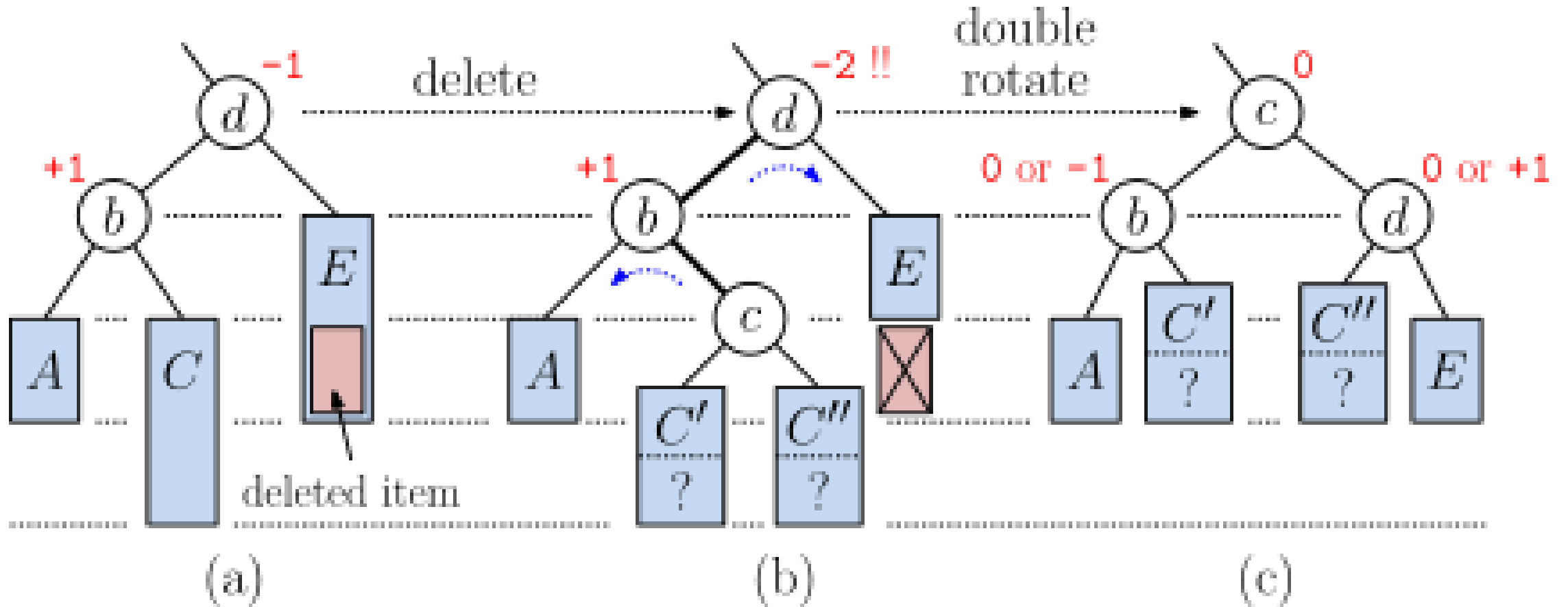
Rebalancing after Insert (Double Rotation)



Rebalancing after Delete (Single Rotation)



Rebalancing after Delete (Double Rotation)



Other Tree-Based Data Structures

There are many variants of balanced binary trees, differing in when/how they rebalance.

Most popular are **Red-Black** trees: taller than AVL trees, but still $\mathcal{O}(\log n)$ height.

Their strategy gives *marginally* slower lookup and *allegedly* faster insert/delete.

Many data structures are based on trees, often **augmented with extra data** to make certain operations fast.

In AVL trees, the extra data is height/balance information, which makes rebalancing fast.

The **heap** is a clever tree-based data structure that can be used to implement efficient priority queues, sorting and maps...

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

Next time: Heaps